

Universidade de Brasília

Faculdade de Ciências e Tecnologias em Engenharia
Engenharia de Software



Orientação por Objetos (FGA0158)

Trabalho Prático: Cafeteria Geek “Byte & Brew”

Autor:

Arthur Pinheiro Marinho Matrícula: 231011112

Brasília
29 de junho de 2026

Sumário

1	Introdução	2
2	Diagrama de Classes UML	3
3	Associações, Heranças e Polimorfismos	5
3.1	Associações	5
3.2	Dependências	5
3.3	Herança Simples	5
3.4	Polimorfismo por Inclusão	6
3.5	Polimorfismo por Sobrescrita	7
3.6	Polimorfismo por Sobrecarga	7
3.7	Polimorfismo por Coerção	8
3.8	Interface Promocional	9
4	Exceções Customizadas	9
4.1	EstoqueInsuficienteException	9
4.2	PontosInsuficientesException	10
4.3	ValorInvalidoException	11
5	Conclusão	12

1 Introdução

O objetivo deste trabalho é aplicar os conceitos de Orientação a Objetos por meio do desenvolvimento de um projeto em Java. Desse modo, foi criado um software para a cafeteria Byte & Brew, o qual gerencia pedidos, cadastro de clientes e estoque do estabelecimento.

2 Diagrama de Classes UML

O diagrama a seguir representa a estrutura completa do sistema, evidenciando as relações de herança, implementação de interface e associação entre as classes modeladas.

3 Associações, Heranças e Polimorfismos

3.1 Associações

Uma associação é definida quando, entre duas classes, uma delas mantém uma referência à outra por meio de um atributo, permitindo que uma classe colabore com a outra durante a execução do sistema, sem que exista entre elas uma relação de herança.

A classe `BancoDeDados` associa-se tanto com `Produto` quanto com `Cliente`, mantendo internamente as listas do cardápio e dos clientes cadastrados. É por meio dessas associações que o sistema centraliza todas as operações de CRUD do estabelecimento.

A classe `Pedido` associa-se com `ItemPedido` e com `Cliente`, centralizando os produtos escolhidos pelo cliente e os dados de quem está realizando a compra.

Por sua vez, `ItemPedido` associa-se com `Produto` para referenciar qual item do cardápio foi solicitado e em qual quantidade.

Logo, essas associações são fundamentais para o sistema, permitindo que pedidos, clientes e produtos colaborem entre si de forma estruturada durante todo o fluxo de venda.

3.2 Dependências

Dependência é a relação em que uma classe utiliza outra de forma temporária, sem mantê-la como atributo, geralmente por meio de parâmetros de método, variáveis locais ou declarações de exceções lançadas.

A classe `App` apresenta dependências com `BancoDeDados` e `Pedido`, sendo o ponto de entrada do sistema. No método `main()`, `App` instancia o `BancoDeDados` para gerenciar os dados em memória e cria objetos `Pedido` conforme o fluxo de vendas é executado.

Por fim, as classes `Pedido` e `Produto` possuem dependência com `EstoqueInsuficienteException`, lançada quando a quantidade solicitada ultrapassa o estoque disponível. `Pedido` e `ClienteVIP` dependem de `PontosInsuficientesException`, lançada quando o saldo de XP é insuficiente para cobrir o valor da compra. Por último, `Produto` e `Comida` dependem de `ValorInvalidoException`, lançada quando um valor negativo é fornecido para atributos como preço, estoque ou tempo de preparo.

Infere-se, portanto, que essas relações não representam vínculos duradouros entre os objetos, mas usos restritos à execução de métodos específicos.

3.3 Herança Simples

A herança simples é utilizada para promover o reuso de código no sistema. Por meio dela, atributos e métodos definidos em uma classe base são automaticamente herdados pelas subclasses, evitando repetição de código e estabelecendo uma hierarquia entre os tipos.

A classe `Produto` é abstrata e define os atributos comuns a todo item do cardápio: `nome`, `precoBase`, `codigo` e `quantidadeEstoque`. Além disso, implementa os métodos *getters* e *setters* desses atributos, bem como os métodos `adicionarEstoque()` e `removerEstoque()`. Por ser abstrata, `Produto` não pode ser instanciada diretamente, obrigando o uso das subclasses concretas. As classes `Bebida` e `Comida` herdam todos esses atributos e métodos, acrescentando cada uma seus próprios atributos específicos de categoria.

De forma similar, a classe `Cliente` é abstrata e define os atributos comuns `nome`, `cpf` e `saldoXP`, compartilhados por todos os clientes cadastrados. As classes `ClienteStandard` e `ClienteVIP` são suas subclasses concretas, cada uma definindo seu próprio multiplicador de pontos XP. Além disso, a classe `ClienteVIP` estende essa base com comportamentos exclusivos, implementando os métodos `pagarComPontos()` e `pontosGastos()`, ausente na classe `ClienteStandard`.

Portanto, em ambos os casos, a herança evita a duplicação de atributos e métodos comuns, concentrando na classe base tudo o que é compartilhado e deixando para as subclasses apenas o que as diferencia.

3.4 Polimorfismo por Inclusão

O polimorfismo por inclusão ocorre quando uma variável do tipo de uma classe base referencia objetos de suas subclasses, permitindo que o sistema os processe de forma genérica e uniforme, sem necessidade de verificar o tipo concreto do objeto.

No sistema, a classe `BancoDeDados` armazena todos os itens do cardápio em uma lista `ArrayList<Produto>`, que pode conter tanto objetos do tipo `Bebida` quanto do tipo `Comida`. No método `acharProduto()`, a variável `produto` é declarada com o tipo genérico `Produto`, mas cada elemento percorrido pode ser uma instância de `Bebida` ou de `Comida`:

```
1 public Produto acharProduto(String codigo){
2     for (Produto produto : cardapio){
3         if (produto.getCodigo().equals(codigo)){
4             return produto;
5         }
6     }
7     return null;
8 }
```

Listing 1: Polimorfismo por inclusão em `acharProduto()`

Da mesma forma, a classe `Pedido` armazena os itens da comanda em uma `ArrayList<ItemPedido>`, onde cada `ItemPedido` mantém uma referência do tipo `Produto`. Desse modo, no método `finalizarPedido()`, essa referência genérica é utilizada para descontar do estoque a quantidade vendida de cada produto, chamando `removeEstoque()` diretamente sobre o tipo `Produto`:

```
1 for (ItemPedido item : itens) {
2     if (item != null) {
3         item.getProduto().removeEstoque(item.getQuantidade());
4     }
5 }
```

Listing 2: Polimorfismo por inclusão em `finalizarPedido()`

Em suma, em nenhum dos dois métodos há necessidade de verificar o tipo concreto do objeto, o que demonstra como o polimorfismo por inclusão simplifica o código responsável por manipular coleções heterogêneas de produtos.

3.5 Polimorfismo por Sobrescrita

O polimorfismo por sobrescrita ocorre quando uma classe base define um método e suas subclasses o reimplementam com o mesmo nome e os mesmos parâmetros de entrada, mas com comportamento distinto. Dessa forma, o tipo de retorno deve ser o mesmo em todas as versões.

No projeto, isso é observado no método abstrato `calcularPontos()`, definido na classe base `Cliente` e sobrescrito em `ClienteStandard` e `ClienteVIP`. Ambas as subclasses utilizam a mesma lógica de cálculo, mas com valores distintos para a constante `MULTIPLICADOR_COMPRA`: 1 em `ClienteStandard` e 2 em `ClienteVIP`, assim refletindo as diferentes taxas de acúmulo de XP do programa de fidelidade.

```
1 // ClienteStandard
2 // MULTIPLICADOR_COMPRA = 1
3 @Override
4 public int calcularPontos(double valorCompra) {
5     return (int) (valorCompra * MULTIPLICADOR_COMPRA);
6 }
7
8 // ClienteVIP
9 // MULTIPLICADOR_COMPRA = 2
10 @Override
11 public int calcularPontos(double valorCompra) {
12     return (int) (valorCompra * MULTIPLICADOR_COMPRA);
13 }
```

Listing 3: Sobrescrita de `calcularPontos()` nas subclasses de `Cliente`

Apesar de o código ser estruturalmente idêntico nas duas subclasses, o resultado é diferente, o que define um exemplo de polimorfismo por sobrescrita.

3.6 Polimorfismo por Sobrecarga

O polimorfismo por sobrecarga ocorre quando uma classe define dois ou mais métodos com o mesmo nome, mas com parâmetros de entrada distintos. O compilador identifica qual versão do método deve ser chamada com base nos argumentos fornecidos.

No projeto, a sobrecarga é aplicada no método `adicionarItem()` da classe `Pedido`, que possui duas assinaturas. A primeira recebe um `Produto` e uma quantidade inteira, permitindo adicionar múltiplas unidades de um mesmo item ao pedido. A segunda recebe apenas um `Produto`, adicionando automaticamente uma unidade, e delega sua execução para a primeira versão do método:

```
1 public void adicionarItem(Produto produto, int quantidade)
2     throws EstoqueInsuficienteException {
3     if (quantidade > produto.getQuantidadeEstoque()) {
4         throw new EstoqueInsuficienteException(
5             "Estoque insuficiente para o produto: " + produto.getNome()
6         );
7     }
8     itens.add(new ItemPedido(produto, quantidade));
9 }
10 public void adicionarItem(Produto produto)
11     throws EstoqueInsuficienteException {
```



```

12     adicionarItem(produto, 1);
13 }

```

Listing 4: Sobrecarga de adicionarItem() em Pedido

O polimorfismo por sobrecarga é aplicado nos construtores da classe `Pedido`. A primeira versão recebe o nome do atendente e um objeto `Cliente`, utilizada quando o cliente possui cadastro no programa de fidelidade. A segunda recebe apenas o nome do atendente, utilizada para compras de clientes casuais, que não possuem vínculo com nenhum cadastro:

```

1 public Pedido(String atendente, Cliente cliente) {
2     this.id = contador++;
3     this.atendente = atendente;
4     this.cliente = cliente;
5     this.itens = new ArrayList<>();
6 }
7
8 public Pedido(String atendente) {
9     this.id = contador++;
10    this.atendente = atendente;
11    this.itens = new ArrayList<>();
12 }

```

Listing 5: Sobrecarga de construtores em Pedido

Em síntese, em ambos os casos, tanto na sobrecarga de `adicionarItem()` quanto nos construtores de `Pedido`, o compilador escolhe a versão correta do método com base nos argumentos fornecidos, evitando a necessidade de parâmetros opcionais ou verificações condicionais para tratar cada cenário separadamente.

3.7 Polimorfismo por Coerção

O polimorfismo por coerção ocorre quando um objeto é convertido para um tipo mais específico, operação chamada de downcast. Quando convertido para um tipo mais genérico, é chamado de upcast.

No projeto, o polimorfismo por coerção foi utilizado no método `calcularValorTotal(int desconto)` dentro da classe `Pedido`, onde a referência do tipo `Produto` é convertida para a interface `Promocional` a fim de acessar o método `aplicarDesconto()`, que não existe na classe base `Produto`. Dessa forma, apenas as bebidas, as quais implementam `Promocional`, recebem o desconto, enquanto as comidas são calculadas pelo preço normal:

```

1 if (objeto.getProduto() instanceof Promocional) {
2     Promocional itemComPromocao = (Promocional) objeto.getProduto();
3     double valorComDesconto = itemComPromocao.aplicarDesconto(desconto);
4     valorTotal += valorComDesconto * objeto.getQuantidade();
5 } else {
6     valorTotal += objeto.getProduto().getPreco() * objeto.getQuantidade();
7 }

```

Listing 6: Downcast de Produto para Promocional em calcularValorTotal()

O outro caso onde ocorre é no método `finalizarPedido()` na classe `Pedido`, onde a referência do tipo `Cliente` é convertida para `ClienteVIP` a fim de acessar os métodos `pagarComPontos()` e `pontosGastos()`, exclusivos dessa subclasse e inexistentes na classe base `Cliente`:

```
1 if (cliente instanceof ClienteVIP) {
2     ClienteVIP clienteVIP = (ClienteVIP) cliente;
3     if (clienteVIP.pagarComPontos(valorTotal)) {
4         clienteVIP.debitarXP(clienteVIP.pontosGastos(valorTotal));
5         pagamentoRealizado = true;
6     }
7 }
```

Listing 7: Downcast de `Cliente` para `ClienteVIP` em `finalizarPedido()`

Em ambos os casos, o `instanceof` garante que a conversão só ocorre quando o objeto é de fato do tipo esperado, tornando o downcast seguro em tempo de execução e evitando erros de `ClassCastException`.

3.8 Interface Promocional

A interface `Promocional`, definida no pacote `br.edu.cafeteria.servico`, estabelece o contrato para aplicação de descontos em dias de evento. Ela declara um único método:

```
1 public interface Promocional {
2     double aplicarDesconto(int desconto);
3 }
```

Listing 8: Interface `Promocional`

No projeto, apenas a classe `Bebida` implementa essa interface, pois segundo as regras de negócio da cafeteria, os descontos em dias de evento se aplicam exclusivamente às bebidas. A classe `Comida` não implementa `Promocional`, de modo que seus itens são sempre calculados pelo preço cheio, independentemente de promoções ativas.

4 Exceções Customizadas

4.1 `EstoqueInsuficienteException`

A `EstoqueInsuficienteException` é uma exceção do tipo *checked*, ou seja, estende diretamente a classe `Exception`. Isso significa que o compilador obriga que todo método que a lance declare explicitamente o `throws` em sua assinatura, e que todo código que chame esses métodos trate ou propague a exceção. Essa escolha é justificada pelo fato de o esgotamento de estoque ser uma situação previsível e esperada no contexto de uma cafeteria, devendo ser tratada de forma explícita pelo sistema.

No projeto, ela é lançada em dois momentos distintos. O primeiro é no método `adicionarItem()` da classe `Pedido`, quando a quantidade solicitada pelo cliente é maior do que a disponível no estoque do produto:

```
1 if (quantidade > produto.getQuantidadeEstoque()) {
2     throw new EstoqueInsuficienteException(
3         "Estoque insuficiente para o produto: " + produto.getNome());
4 }
```

```
4 }
```

Listing 9: Lançamento em adicionarItem() de Pedido

O segundo momento é no método `removerEstoque()` da classe `Produto`, chamado internamente durante a finalização do pedido em `finalizarPedido()`, garantindo que o estoque não seja decrementado além do disponível:

```
1 if (quantidade > this.quantidadeEstoque) {  
2     throw new EstoqueInsuficienteException(  
3         "A quantidade a ser removida e maior do que o estoque  
4         disponivel.");  
5 }
```

Listing 10: Lançamento em removerEstoque() de Produto

Em ambos os momentos, a exceção interrompe a operação imediatamente, preservando a consistência dos dados de estoque e impedindo que o sistema processe vendas para as quais não há produtos disponíveis.

4.2 PontosInsuficientesException

A `PontosInsuficientesException` também é uma exceção do tipo *checked*, estendendo diretamente a classe `Exception`. Desse modo, o compilador obriga o tratamento dessa exceção em todo código que chame os métodos que a lançam. Essa escolha é justificada pelo fato de o saldo insuficiente de XP ser uma situação previsível e recorrente no programa de fidelidade, devendo ser tratada de forma controlada pelo sistema.

No projeto, ela é lançada em dois momentos. O primeiro é no método exclusivo `pagarComPontos()` da classe `ClienteVIP`, quando o saldo de XP do cliente é insuficiente para cobrir o valor total da compra:

```
1 public boolean pagarComPontos(double valorCompra)  
2     throws PontosInsuficientesException {  
3     int pontosNecessarios = (int) (valorCompra * MULTIPLICADOR_RESGATE  
4     );  
5     if (getSaldoXP() >= pontosNecessarios) {  
6         return true;  
7     } else {  
8         throw new PontosInsuficientesException(  
9             "Saldo de XP insuficiente para realizar a compra.");  
10    }  
11 }
```

Listing 11: Lançamento em pagarComPontos() de ClienteVIP

O segundo momento é no método `finalizarPedido()` da classe `Pedido`, que propaga a exceção lançada por `pagarComPontos()` ao código chamador, permitindo que a classe `App` trate o erro e ofereça ao cliente a opção de realizar o pagamento de forma convencional:

```
1 try {  
2     boolean finalizado = pedido.finalizarPedido(querPagarComPontos,  
3     desconto);  
4     if (finalizado) {  
5         System.out.println(pedido.toString());  
6     }  
7 }
```

```

6 } catch (PontosInsuficientesException e) {
7     System.out.println("Erro: " + e.getMessage());
8     System.out.println("Realizando pagamento normal...");
9     pedido.finalizarPedido(false, desconto);
10 }

```

Listing 12: Tratamento em finalizarPedido() de Pedido

Com isso, o sistema garante que nenhum cliente VIP consiga pagar com pontos sem saldo suficiente, mantendo a integridade do programa de fidelidade e oferecendo ao cliente a alternativa do pagamento convencional.

4.3 ValorInvalidoException

A `ValorInvalidoException` é uma exceção do tipo *unchecked*, estendendo `RuntimeException`. Dessa forma, o compilador não obriga o seu tratamento explícito. Essa escolha é justificada pelo fato de ela representar um erro de programação — a passagem de um valor negativo para um atributo que não o aceita — e não uma situação de negócio esperada. Em outras palavras, se essa exceção for lançada, indica que o código chamador cometeu um erro que deve ser corrigido, não contornado.

No projeto, ela é lançada nos *setters* de validação das classes `Produto` e `Comida`, sempre que um valor inválido é fornecido para atributos como preço, quantidade em estoque ou tempo de preparo:

```

1 // Produto
2 public void setPreco(double preco) {
3     if (preco < 0) {
4         throw new ValorInvalidoException(
5             "0 preco nao pode ser negativo.");
6     }
7     this.precoBase = preco;
8 }
9
10 // Comida
11 public void setTempoPreparoMin(int tempoPreparoMin) {
12     if (tempoPreparoMin < 0) {
13         throw new ValorInvalidoException(
14             "0 tempo de preparo nao pode ser negativo.");
15     }
16     this.tempopreparoMin = tempoPreparoMin;
17 }

```

Listing 13: Lançamento em setters de Produto e Comida

Por ser *unchecked*, sua ocorrência sinaliza diretamente uma inconsistência no código, tornando desnecessário o tratamento obrigatório em todos os pontos que chamam esses métodos. Diferentemente das exceções de negócio, que tratam situações esperadas, a `ValorInvalidoException` serve como uma barreira de proteção contra o uso incorreto das classes, garantindo que atributos críticos nunca recebam valores inválidos.

5 Conclusão

O desenvolvimento deste trabalho permitiu a integração dos conceitos de Orientação a Objetos por meio de um software em Java. O projeto implementou herança, polimorfismo e suas variações, encapsulamento, interfaces e exceções customizadas.

Entretanto, o sistema ainda possui limitações, principalmente o fato de os dados serem armazenados apenas em memória, sendo perdidos a cada execução do programa.

Mesmo assim, a arquitetura modular e encapsulada do projeto permite que novas funcionalidades sejam incorporadas sem a necessidade de alterações estruturais, bastando estender as classes ou implementar as interfaces já existentes.